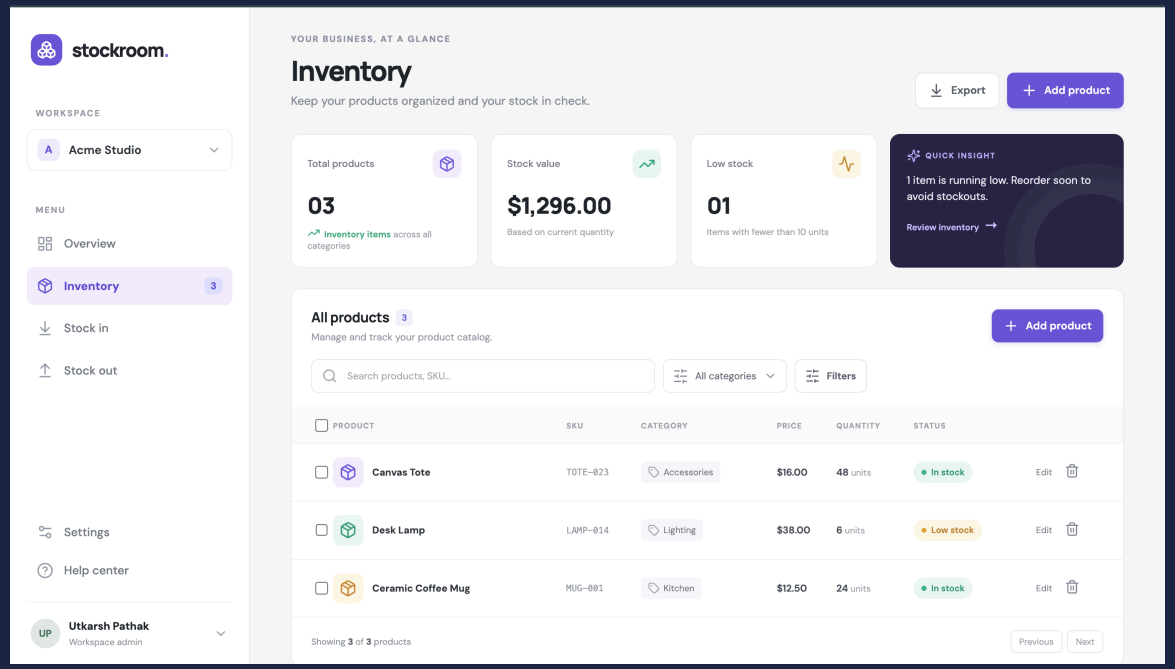
FINAL STUDENT PROJECT

Stockroom

Inventory management, shipped with a DevOps pipeline.

A full-stack CRUD application with automated testing, secure image delivery, and local Kubernetes operations.

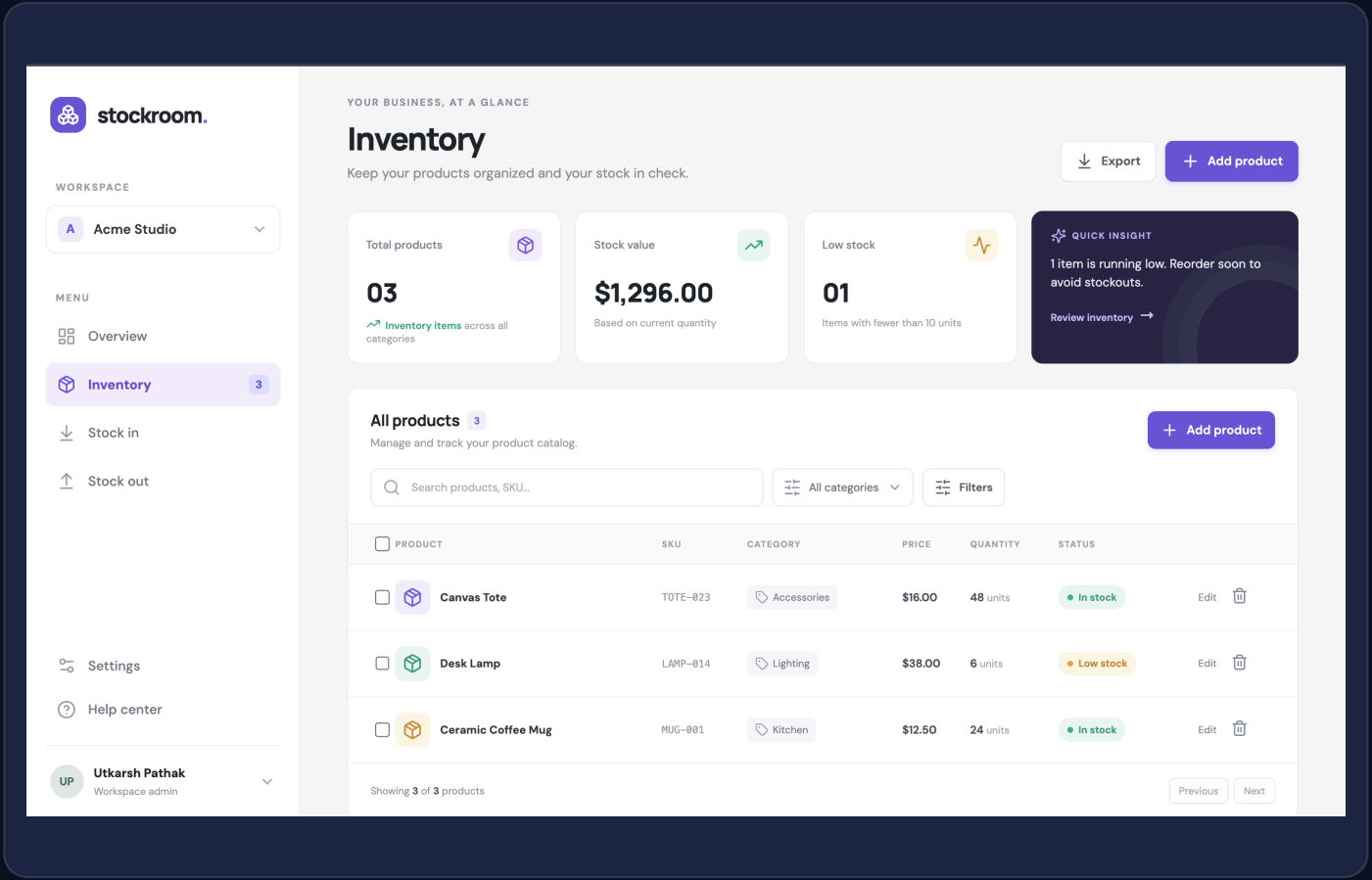
React | FastAPI | PostgreSQL | Kind + Helm



Built for teams that need a clear view of stock.

An inventory workspace made for daily operations

The dashboard keeps catalog changes, stock risk, and inventory value in one view.



CORE WORKFLOWS

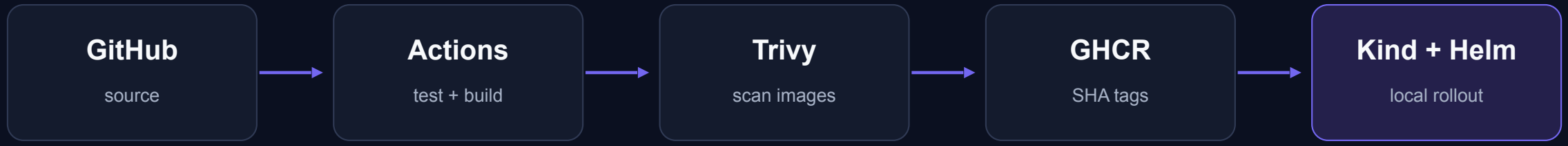
- Create and maintain product records
- Search, filter, and review the catalog
- Track quantities, pricing, and low-stock items
- Export inventory data as CSV

RESPONSIVE FRONTEND

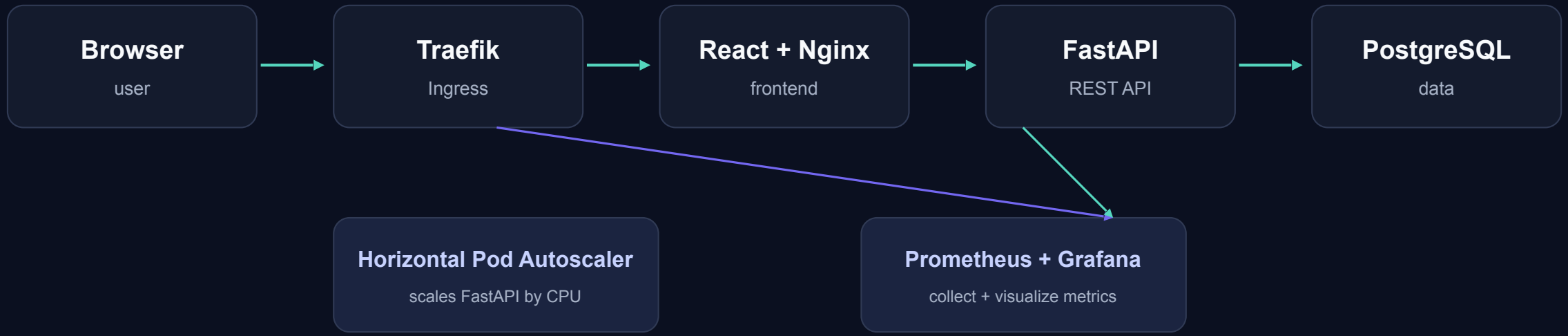
React + Vite, served through Nginx

One local stack connects the app, delivery pipeline, and monitoring

The application and the image pipeline run locally or on GitHub Actions; no AWS resources are used.



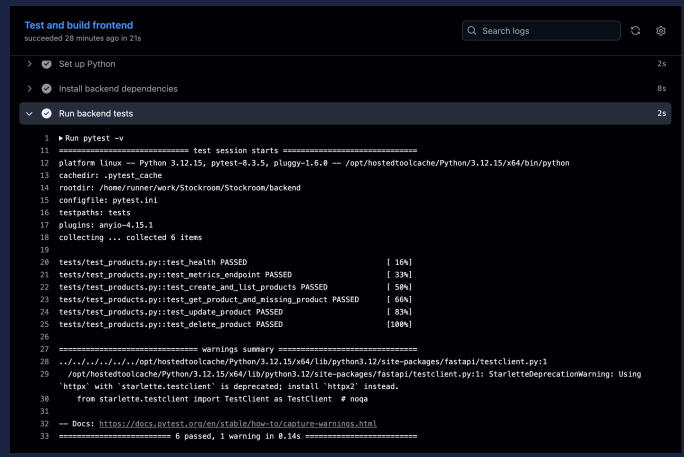
REQUEST PATH



A small REST API keeps the CRUD contract clear

FastAPI, PostgreSQL, SQLAlchemy, and Alembic support the inventory lifecycle.

METHOD	ROUTE	PURPOSE
GET	/api/products	list
GET	/api/products/{id}	detail
POST	/api/products	create
PUT	/api/products/{id}	update
DELETE	/api/products/{id}	delete



AUTOMATED CHECKS

Six API tests cover product operations and metrics.

Tests use an isolated SQLite fixture; they do not depend on the local Postgres service.

Every main-branch change can produce a traceable release

Tests and the frontend build gate the image build, scan, and publish stage.

01 Commit
GitHub main

→

02 Verify
Pytest + Vite build

→

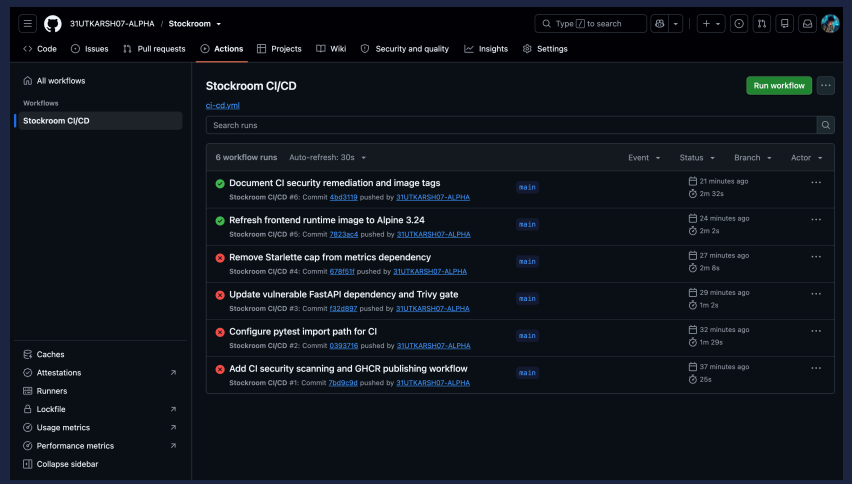
03 Build
Backend + frontend

→

04 Scan
Trivy

→

05 Publish
GHCR + source SHA



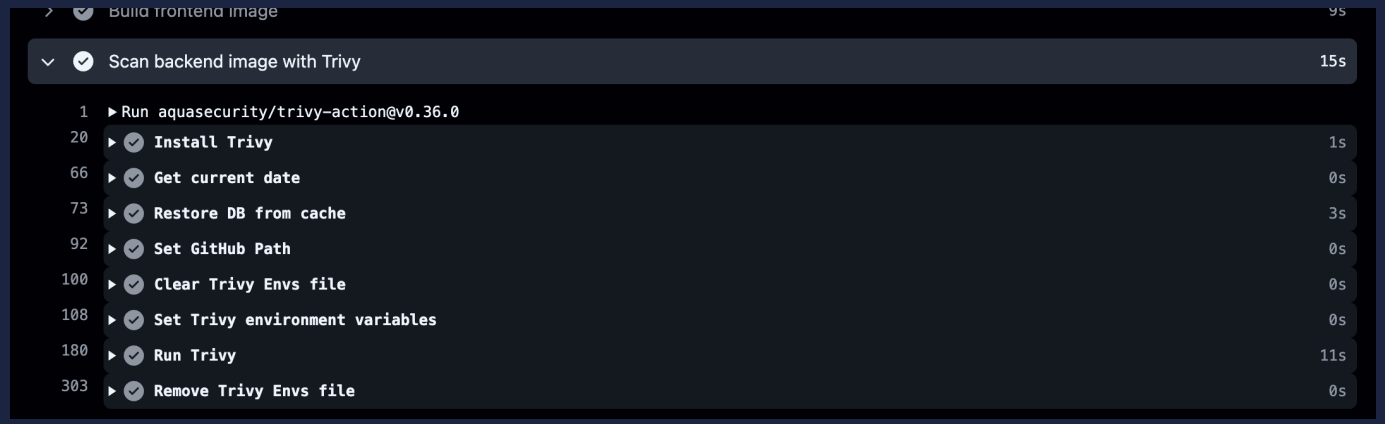
RELEASE TRACEABILITY

One source revision, two images

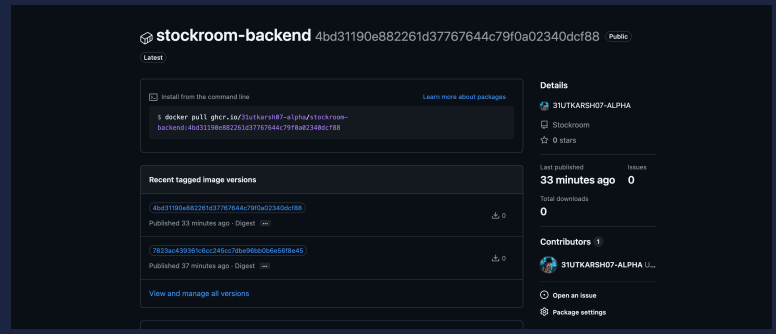
- Backend and frontend images published to GHCR
- Full commit SHA used as each image tag

Image scanning is part of the release gate

Trivy checks both application images before the pipeline publishes them.







POLICY

Actionable HIGH and CRITICAL findings block publishing; image tags retain source-SHA traceability.

Kind and Helm make the deployment reproducible without cloud spend

Traefik routes app traffic; PostgreSQL and the application run inside the local cluster.

```
utkarshpathak3107@Utkarshs-MacBook-Air terraform %    kubectl get pods,svc,ingress -n stockroom -o wide
NAME                                      READY   STATUS    RESTARTS      AGE   IP            NODE                    NOMINATED NODE   READINESS GATES
pod/stockroom-backend-55d65cf4bf-ncthn    1/1     Running   3 (12m ago)   12m   10.244.0.10   stockroom-control-plane   <none>           <none>
pod/stockroom-backend-55d65cf4bf-vc9wf    1/1     Running   3 (12m ago)   12m   10.244.0.6    stockroom-control-plane   <none>           <none>
pod/stockroom-frontend-6c478c77d7-vvlnz   1/1     Running   5 (11m ago)   12m   10.244.0.7    stockroom-control-plane   <none>           <none>
pod/stockroom-frontend-6c478c77d7-wx74p   1/1     Running   5 (11m ago)   12m   10.244.0.9    stockroom-control-plane   <none>           <none>
pod/stockroom-postgresql-59bcb899df-4b6hv 1/1     Running   0             12m   10.244.0.8    stockroom-control-plane   <none>           <none>

NAME                          TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)    AGE   SELECTOR
service/backend               ClusterIP   10.96.91.84    <none>        8000/TCP   10m   app.kubernetes.io/component=backend,app.kubernetes.io/instance=stockroom
service/stockroom-backend     ClusterIP   10.96.43.2     <none>        8000/TCP   12m   app.kubernetes.io/component=backend,app.kubernetes.io/instance=stockroom
service/stockroom-frontend    ClusterIP   10.96.155.23   <none>        80/TCP     12m   app.kubernetes.io/component=frontend,app.kubernetes.io/instance=stockroom
service/stockroom-postgresql  ClusterIP   10.96.58.173   <none>        5432/TCP   12m   app.kubernetes.io/component=database,app.kubernetes.io/instance=stockroom

NAME                                CLASS     HOSTS   ADDRESS   PORTS   AGE
ingress.networking.k8s.io/stockroom traefik   *                 80      12m
```

```
utkarshpathak3107@Utkarshs-MacBook-Air terraform % helm status stockroom -n stockroom
NAME: stockroom
LAST DEPLOYED: Fri Oct 16 01:00:56 2026
NAMESPACE: stockroom
STATUS: deployed
REVISION: 2
DESCRIPTION: Upgrade complete
RESOURCES:
==> v1/Secret
NAME                  TYPE     DATA   AGE
stockroom-postgresql  Opaque   4      14m

==> v1/Service
NAME                TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)    AGE
backend             ClusterIP   10.96.91.84    <none>        8000/TCP   11m
stockroom-backend   ClusterIP   10.96.43.2     <none>        8000/TCP   14m
stockroom-frontend  ClusterIP   10.96.155.23   <none>        80/TCP     14m
stockroom-postgresql  ClusterIP   10.96.58.173   <none>        5432/TCP   14m

==> v1/Deployment
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
stockroom-backend   2/2     2            2           14m
stockroom-frontend  2/2     2            2           14m
stockroom-postgresql  1/1   1            1           14m

==> v1/Pod(related)
NAME                                 READY   STATUS    RESTARTS      AGE
stockroom-backend-55d65cf4bf-ncthn   1/1     Running   3 (13m ago)   14m
stockroom-backend-55d65cf4bf-vc9wf   1/1     Running   3 (13m ago)   14m
stockroom-frontend-6c478c77d7-vvlnz  1/1     Running   5 (12m ago)   14m
stockroom-frontend-6c478c77d7-wx74p  1/1     Running   5 (12m ago)   14m
stockroom-postgresql-59bcb899df-4b6hv  1/1   Running   0             14m

==> v1/Ingress
NAME        CLASS     HOSTS   ADDRESS   PORTS   AGE
stockroom   traefik   *                 80      14m


TEST SUITE: None
```

LOCAL CLUSTER

Kind on Docker Desktop

PACKAGE MANAGER

Stockroom Helm chart

TRAFFIC

Traefik Ingress at 127.0.0.1:8080

CLOUD SCOPE

No AWS resources created

The backend scales from two to five pods on CPU

Metrics Server feeds an autoscaling/v2 HPA targeting average CPU utilization.

CPU TARGET

50%

of requested CPU

REPLICA FLOOR

2

minimum replicas

REPLICA CEILING

5

maximum replicas

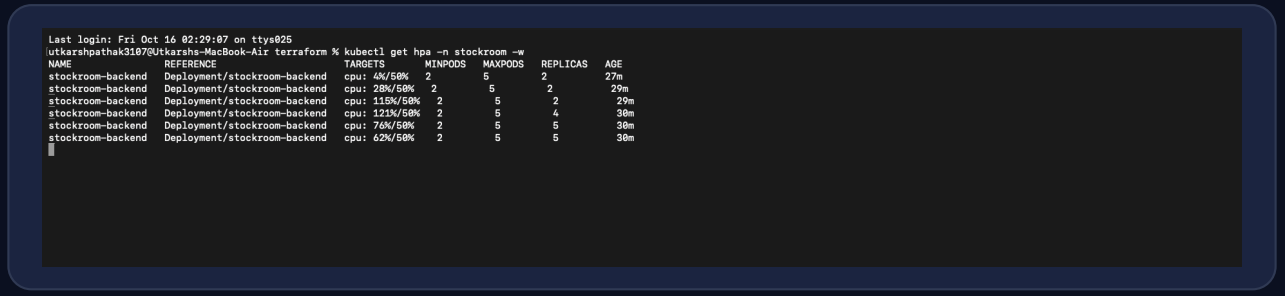
CPU REQUEST

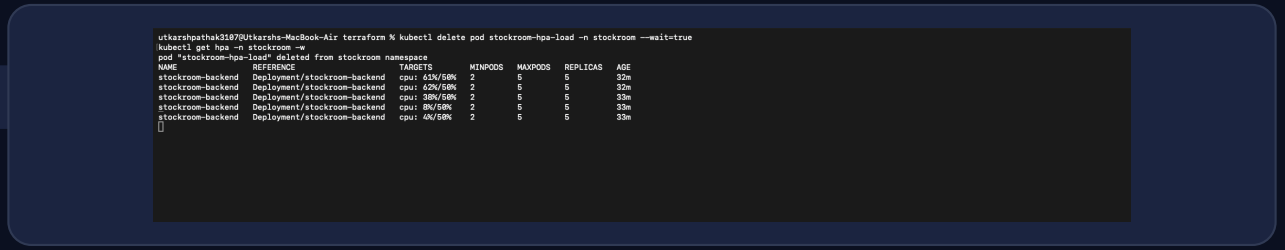
100m

per backend pod

LOAD TEST OBSERVATION

CPU rose above target; the HPA increased desired replicas to five. After load removal, utilization fell and scale-down began.



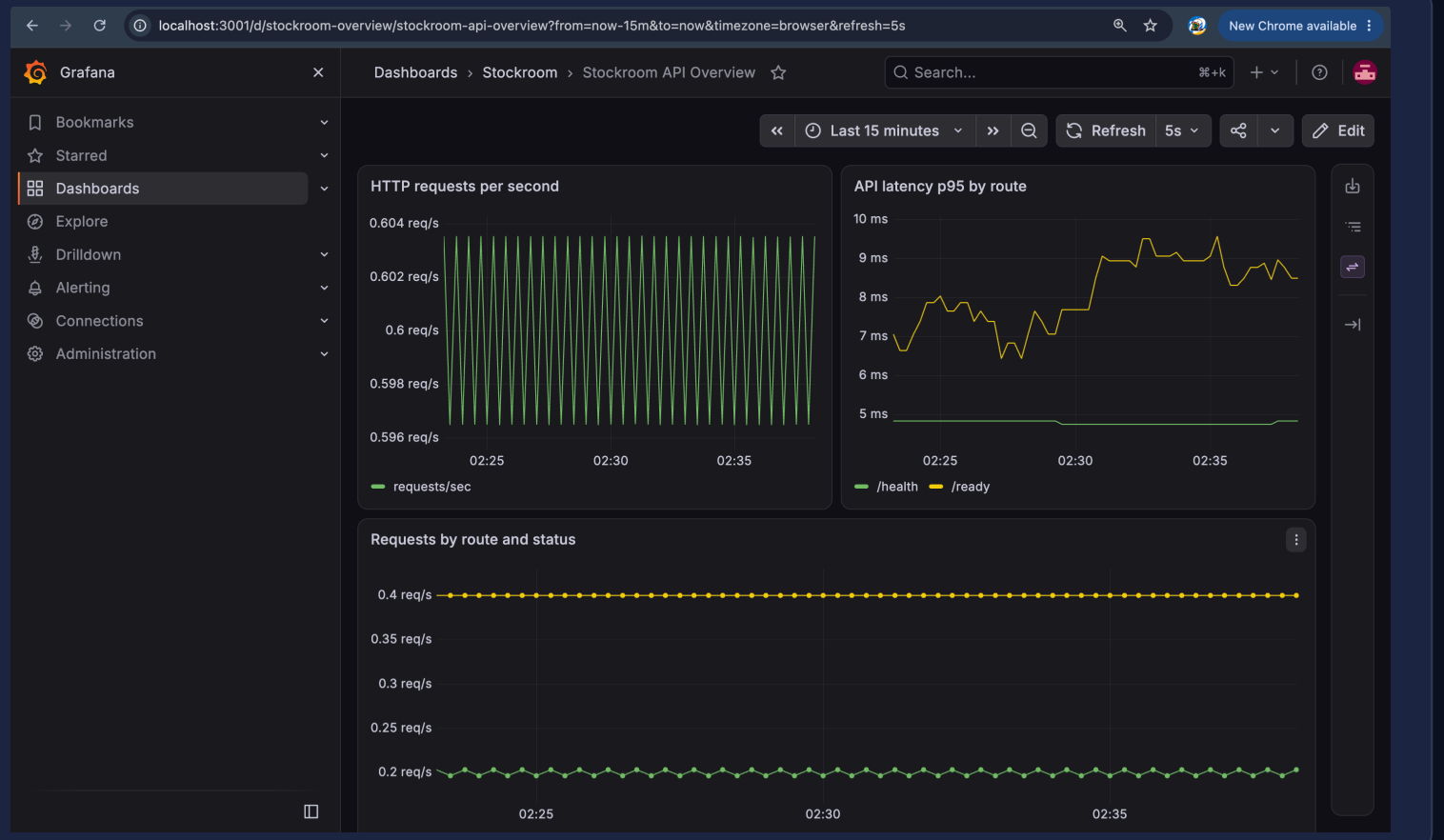


busybox → /api/products

metrics-server → HPA

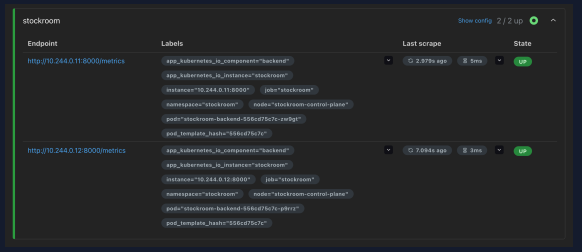
Metrics turn runtime behavior into something operators can inspect

Prometheus scrapes the API; Grafana displays request rate and route latency.



SIGNALS

- HTTP requests per second
- API latency by route
- Requests by route and status



Prometheus target health

The project covers the full application-to-operations path

Final-project minimum requirements are mapped to working code and repository evidence.

APPLICATION

- React responsive frontend
- FastAPI REST API + PostgreSQL
- Product CRUD + CSV export

ENGINEERING

- Git + GitHub Actions
- Pytest + Docker
- Trivy + GHCR SHA tags

DEVOPS

- Kind + Helm + Ingress
- CPU HPA + metrics-server
- Prometheus + Grafana

SCOPE NOTE

Terraform is included as an AWS VPC/EKS reference, but no AWS plan or cloud resources were created.

Reproduction steps, deployment commands, screenshots, and the local walkthrough are in the repository.

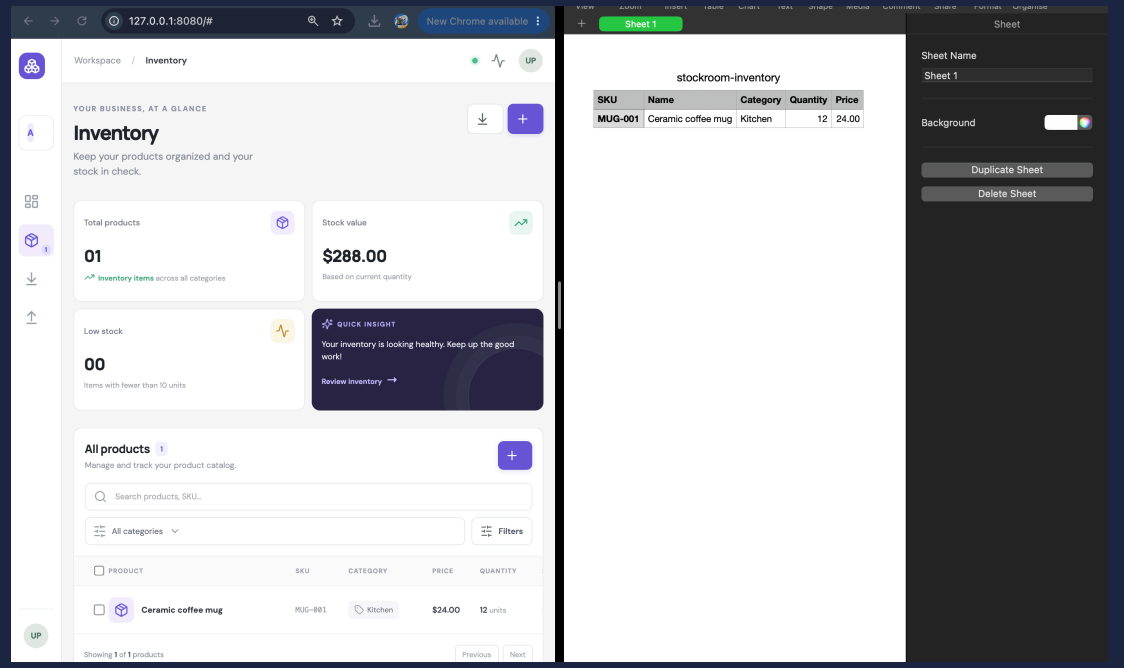
STOCKROOM

From product record to observable service.

A practical inventory app backed by a tested API and a reproducible, locally operated delivery stack.

Source repository

github.com/31UTKARSH07-ALPHA/Stockroom



Thanks